

Junnior Marcellino Polla

11231034

Sistem Paralel dan terdistribusi B

UAS

Link github:

<https://github.com/junN0ir/-UAS-Pub-Sub-Log-Aggregator-Terdistribusi>

Link Youtube:

<https://youtu.be/tngRHQBZIs>

Ringkasan Sistem

Sistem yang dibangun adalah Pub-Sub Log Aggregator Terdistribusi berbasis multi-service yang berjalan menggunakan Docker Compose. Sistem ini menggunakan Python FastAPI sebagai service aggregator, publisher sebagai simulator pengirim event, PostgreSQL sebagai penyimpanan persisten, dan Redis sebagai broker internal atau service pendukung. Semua service berjalan pada jaringan lokal Docker Compose tanpa ketergantungan pada layanan eksternal publik.

Tujuan utama sistem ini adalah menerima event dari publisher, memvalidasi format event, menyimpan event unik, menolak event duplikat, mencatat audit log, dan menyediakan statistik sistem. Sistem mendukung idempotency, yaitu event yang sama tetap hanya diproses satu kali walaupun dikirim berulang. Identitas unik event ditentukan berdasarkan kombinasi topic dan event_id.

Dalam teori sistem terdistribusi, komponen-komponen yang berada pada komputer atau proses berbeda berkomunikasi melalui pertukaran pesan. sistem terdistribusi memiliki karakteristik penting seperti konkurensi antar komponen, tidak adanya global clock yang sempurna, dan kemungkinan kegagalan independen pada tiap komponen. Sistem ini menerapkan konsep tersebut melalui service publisher, aggregator, storage, dan broker yang berjalan sebagai komponen terpisah namun saling berkomunikasi melalui jaringan Compose.

```

pubsub-aggregator/
├── aggregator/
│   ├── Dockerfile
│   ├── main.py          # FastAPI app + endpoints
│   ├── database.py      # asyncpg pool + query logic
│   ├── schemas.py       # Pydantic models
│   └── requirements.txt
├── publisher/
│   ├── Dockerfile
│   ├── main.py          # Event generator + HTTP sender
│   └── requirements.txt
├── init-db/
│   └── init.sql          # DDL schema PostgreSQL
├── tests/
│   ├── __init__.py
│   ├── test_api.py       # Endpoint API (8 tests)
│   ├── test_dedup.py     # Idempotency & dedup (5 tests)
│   ├── test_schema.py    # Validasi schema (7 tests)
│   ├── test_stats.py     # Konsistensi statistik (4 tests)
│   ├── test_persistence.py # Persistensi data (3 tests)
│   ├── test_concurrency.py # Race condition (3 tests)
│   └── test_stress.py    # Load test 20k event (3 tests)
├── docker-compose.yml
└── README.md

```

Arsitektur Sistem

Arsitektur sistem terdiri dari empat service utama:

1. **publisher**

Service ini berperan sebagai producer atau simulator pengirim event. Publisher mengirim event ke aggregator, termasuk event duplikat untuk menguji kemampuan deduplication.

2. **aggregator**

Service utama berbasis FastAPI yang menerima event melalui endpoint HTTP. Endpoint utama yang tersedia adalah /publish, /publish/batch, /events, /stats, /audit, dan /health.

3. **storage / PostgreSQL**

Database persisten untuk menyimpan event unik, audit log, outbox, dan statistik. PostgreSQL menggunakan named volume sehingga data tetap aman walaupun container dihapus atau dibuat ulang.

4. **broker / Redis**

Redis digunakan sebagai service internal dalam jaringan Compose. Redis memperkuat rancangan multi-service dan dapat digunakan sebagai broker atau komponen pendukung komunikasi.

Arsitektur ini mengikuti pendekatan distributed system karena setiap service memiliki peran berbeda dan berkomunikasi melalui jaringan. Salah satu tantangan sistem terdistribusi adalah heterogeneity, concurrency, scalability, failure handling, dan resource sharing. Pada sistem ini, resource utama berupa event dan database dikelola secara terpisah, sedangkan komunikasi antar service dilakukan melalui HTTP dan jaringan internal Docker Compose.

@KamranServices

services:

▷Run Service

storage:

```
image: postgres:16-alpine
container_name: pg_storage
restart: unless-stopped
environment:
  POSTGRES_DB: logdb
  POSTGRES_USER: loguser
  POSTGRES_PASSWORD: logpass
volumes:
  - pg_data:/var/lib/postgresql/data
  - ./init-db/init.sql:/docker-entrypoint-initdb.d/init.sql
```

healthcheck:

```
test: ["CMD-SHELL", "pg_isready -U loguser -d logdb"]
interval: 5s
timeout: 5s
retries: 10
```

▷Run Service

broker:

```
image: redis:7-alpine
container_name: redis_broker
restart: unless-stopped
volumes:
  - broker_data:/data
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 5s
  timeout: 3s
  retries: 10
```

```

▷Run Service
broker:
  image: redis:7-alpine
  container_name: redis_broker
  restart: unless-stopped
  volumes:
    - broker_data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 3s
    retries: 10

▷Run Service
aggregator:
  build:
    context: ./aggregator
    dockerfile: Dockerfile
  container_name: aggregator_service
  restart: unless-stopped
  depends_on:
    storage:
      condition: service_healthy
    broker:
      condition: service_healthy
  environment:
    DATABASE_URL: postgresql://loguser:logpass@storage:5432/logdb
    REDIS_URL: redis://broker:6379
    APP_HOST: "0.0.0.0"
    APP_PORT: "8080"
    WORKER_COUNT: "4"
  ports:
    - "8080:8080"
  healthcheck:
    test: ["CMD-SHELL", "curl -f http://localhost:8080/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 5

```

Analisis Performa dan Metrik

Alur proses dimulai dari publisher yang membuat event dalam format JSON. Setiap event memiliki field minimal:

```

{
  "topic": "system.order",
  "event_id": "uuid-unik",
  "timestamp": "2025-01-01T00:00:00Z",
  "source": "service-name",

```

```
"payload": {}  
}
```

Event dikirim ke aggregator melalui endpoint /publish untuk satu event atau /publish/batch untuk banyak event. Setelah diterima, aggregator memvalidasi schema menggunakan Pydantic. Jika data valid, aggregator membuka transaksi database untuk memproses event.

Di dalam database, sistem melakukan insert ke tabel events menggunakan pola INSERT ... ON CONFLICT DO NOTHING. Jika kombinasi topic dan event_id belum ada, event dianggap baru dan disimpan. Jika kombinasi tersebut sudah ada, event dianggap duplikat dan tidak disimpan ulang. Walaupun event duplikat tidak diproses ulang, audit log tetap mencatat request tersebut sebagai bukti observability.

Keputusan Desain

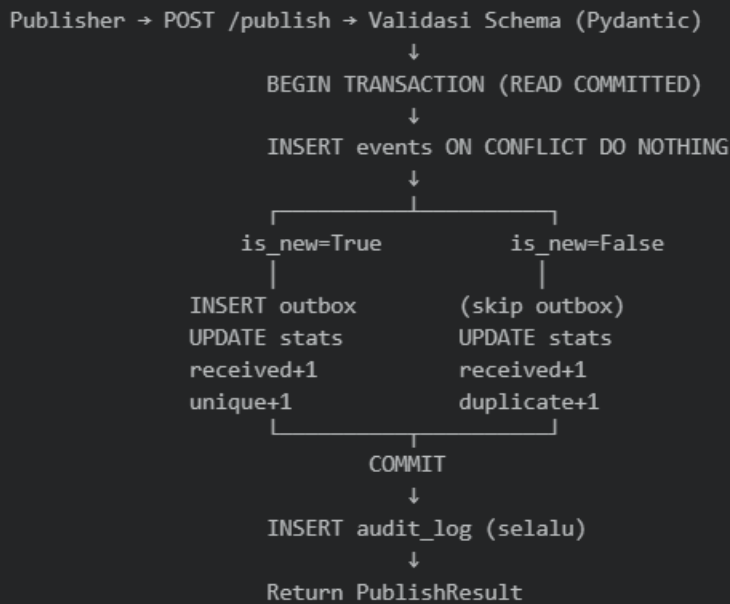
Idempotency dan Deduplication

Idempotency diterapkan dengan menjadikan pasangan topic dan event_id sebagai identitas unik event. Event dengan topic dan event_id yang sama hanya boleh diproses satu kali. Hal ini penting karena pada sistem terdistribusi, publisher dapat mengirim ulang event akibat retry, timeout, atau kegagalan jaringan.

Deduplication dilakukan di level database menggunakan unique constraint pada kombinasi topic dan event_id. Dengan pendekatan ini, keputusan apakah event baru atau duplikat tidak hanya bergantung pada logika aplikasi, tetapi dikunci oleh PostgreSQL. Jika terjadi konflik, query ON CONFLICT DO NOTHING mencegah data duplikat masuk ke tabel event.

Pola ini sesuai dengan kebutuhan idempotent consumer. Concurrency dan independent failures adalah karakteristik utama sistem terdistribusi. Karena itu, consumer harus tetap aman ketika menerima event berulang atau ketika beberapa request masuk hampir bersamaan.

Alur Proses Event



Transaksi dan Kontrol Konkurensi

Setiap event diproses menggunakan transaksi database. Dalam satu transaksi, sistem mencoba menyimpan event, menulis audit log, dan memperbarui statistik. Jika event baru, counter `unique_processed` bertambah. Jika event duplikat, counter `duplicate_dropped` bertambah. Counter `received` selalu bertambah karena semua event yang masuk tetap dihitung.

Kontrol konkurensi dilakukan melalui kombinasi `unique constraint` dan `INSERT ... ON CONFLICT DO NOTHING`. Jika dua request dengan event yang sama masuk secara paralel, PostgreSQL hanya mengizinkan satu insert berhasil. Request lainnya akan terkena conflict dan dihitung sebagai duplicate. Dengan cara ini, sistem mencegah race condition dan double-processing.

Transaksi dan kontrol konkurensi dibahas sebagai mekanisme penting untuk menjaga konsistensi data ketika operasi berjalan bersamaan. Pada implementasi ini, transaksi digunakan untuk menjaga atomicity dan consistency, sedangkan `unique constraint` digunakan untuk menyelesaikan konflik penulisan secara aman.

```
>> '0
(.venv) PS D:\belajar hebat\Sister\uas-pub-sub> Invoke-RestMethod -Uri "http://localhost:8080/publish/batch" -Method POST -ContentType "application/j
son" -Body $batchBody | ConvertTo-Json
{
  "status": "ok",
  "accepted": 2,
  "duplicates": 1,
  "errors": 0
}
(.venv) PS D:\belajar hebat\Sister\uas-pub-sub>
```

Persistensi Data

Persistensi data dijaga menggunakan named volume PostgreSQL. Data event, audit log, dan statistik tidak disimpan di memori aggregator, melainkan di database. Karena itu, ketika container aggregator direstart atau dibuat ulang, data tetap tersedia.

Deduplication tetap bekerja setelah restart karena informasi event yang sudah pernah diproses masih tersimpan di PostgreSQL. Hal ini dibuktikan dengan mengirim ulang event lama setelah aggregator direstart. Sistem tetap mengembalikan response duplicate, bukan accepted.

Observability

Observability disediakan melalui endpoint /health, /stats, /events, dan /audit. Endpoint /health digunakan untuk memeriksa kondisi aggregator. Endpoint /stats menampilkan metrik sistem seperti received, unique_processed, duplicate_dropped, topics, uptime, duplicate rate, dan lag_seconds. Endpoint /events menampilkan event unik yang tersimpan. Endpoint /audit menampilkan riwayat semua event yang masuk, termasuk duplikat.

Observability penting karena dalam sistem terdistribusi, error dapat terjadi pada banyak titik, seperti publisher, jaringan, aggregator, atau database. Dengan log dan endpoint monitoring, kondisi sistem lebih mudah diperiksa saat demo maupun debugging.

```
(.venv) PS D:\belajar hebat\Sister\uas-pub-sub> curl.exe http://localhost:8080/stats
{"received":19531,"unique_processed":15015,"duplicate_dropped":4516,"error_count":0,"topics":["system.audit","system.auth","system.notification","sys
tem.order","system.payment","test.concurrent"],"uptime_seconds":96.38,"duplicate_rate_percent":23.12,"lag_seconds":30.38}
(.venv) PS D:\belajar hebat\Sister\uas-pub-sub>
```

Keterikatan Teori

T1 Karakteristik Sistem Terdistribusi dan Arsitektur

Sistem yang dibangun termasuk sistem terdistribusi karena terdiri dari beberapa service yang berjalan terpisah, yaitu publisher, aggregator, storage PostgreSQL, dan broker Redis. Setiap service memiliki tanggung jawab berbeda dan berkomunikasi melalui jaringan lokal Docker Compose. Van Steen dan Tanenbaum (2023) menjelaskan bahwa sistem terdistribusi berkaitan dengan proses dan resource yang tersebar pada beberapa komputer atau node,

tetapi tetap bekerja sebagai satu sistem. Pada implementasi ini, pengguna cukup mengakses aggregator melalui localhost:8080, sedangkan komunikasi internal antar-service tetap disembunyikan oleh Docker Compose.

Dari sisi arsitektur, sistem memakai pendekatan **publish–subscribe** dan **microservices**. Publisher hanya bertugas mengirim event, sedangkan aggregator menerima dan memproses event. PostgreSQL dipisahkan sebagai storage persisten, dan Redis disiapkan sebagai broker internal. Keputusan ini membuat sistem lebih modular, mudah diuji, dan lebih dekat dengan desain distributed system modern. Trade-off yang diambil adalah sistem menjadi lebih kompleks dibanding aplikasi monolitik, tetapi lebih jelas dalam pemisahan tanggung jawab, observability, dan pengujian kegagalan antar-service (van Steen & Tanenbaum, 2023).

T2 Komunikasi Antar Komponen dan Penamaan

Komunikasi antar komponen dilakukan melalui HTTP request berbasis JSON. Publisher mengirim event ke aggregator melalui endpoint /publish atau /publish/batch. Aggregator kemudian memvalidasi data, memproses event, dan menyimpan hasilnya ke PostgreSQL. Dalam sistem terdistribusi, komunikasi antar proses merupakan aspek utama karena setiap service tidak berbagi memori secara langsung, melainkan bertukar pesan melalui jaringan. Hal ini sesuai dengan pembahasan komunikasi pada distributed systems, yaitu bagaimana proses bertukar data melalui message-oriented communication atau mekanisme request-response (van Steen & Tanenbaum, 2023).

Penamaan event dilakukan menggunakan dua field utama, yaitu topic dan event_id. Field topic digunakan sebagai kategori event, misalnya system.order, system.auth, atau system.payment. Field event_id digunakan sebagai identitas unik event. Kombinasi topic dan event_id menjadi dasar deduplication. Konsep ini berkaitan dengan naming dalam distributed systems, karena setiap objek atau resource perlu memiliki identitas yang dapat digunakan untuk mengenali dan mengaksesnya secara konsisten. Dengan skema ini, sistem dapat membedakan event baru dan event duplikat secara jelas (van Steen & Tanenbaum, 2023).

T3 Waktu dan Ordering

Setiap event memiliki field timestamp dengan format ISO 8601, misalnya 2025-01-01T00:00:00Z. Timestamp digunakan untuk mencatat waktu event dibuat oleh publisher. Selain itu, database juga memiliki waktu penerimaan atau urutan ID internal yang membantu sistem menampilkan event secara stabil. Namun, sistem ini tidak menjanjikan **total ordering global**, karena dalam sistem terdistribusi urutan absolut antar-event sulit dijamin tanpa mekanisme koordinasi tambahan.

Van Steen dan Tanenbaum (2023) menjelaskan konsep logical clocks dan ordering, termasuk gagasan bahwa yang sering lebih penting dalam distributed system bukan waktu fisik yang benar-benar sama, tetapi urutan kejadian yang dapat digunakan untuk memahami hubungan antar-event. Pada sistem ini, ordering yang digunakan bersifat praktis: event dapat diurutkan berdasarkan timestamp dan waktu diterima, tetapi event yang datang terlambat tetap diterima selama valid. Dengan demikian, sistem toleran terhadap event out-of-order dan tetap menjaga fokus utama, yaitu idempotency dan deduplication.

T4 Toleransi Kegagalan

Sistem ini mempertimbangkan beberapa failure modes, seperti event dikirim ulang, request timeout, publisher melakukan retry, aggregator restart, database belum siap, atau container dibuat ulang. Dalam distributed system, kegagalan sebagian merupakan kondisi yang umum karena satu komponen dapat gagal sementara komponen lain tetap berjalan. Oleh karena itu, sistem harus dirancang agar tetap konsisten walaupun terjadi retry atau crash pada salah satu service (van Steen & Tanenbaum, 2023).

Mitigasi yang diterapkan adalah penggunaan healthcheck, retry dari publisher, dedup store persisten, dan graceful restart. Healthcheck memastikan aggregator dan PostgreSQL sudah siap sebelum publisher mengirim event. Dedup store disimpan di PostgreSQL sehingga event lama tetap dikenali setelah restart. Jika publisher mengirim event yang sama akibat retry, aggregator tidak memproses ulang event tersebut karena topic dan event_id sudah tercatat. Dengan strategi ini, sistem tetap aman terhadap duplikasi, crash, dan proses restart selama volume database tidak dihapus.

T5 Konsistensi dan Replikasi

Konsistensi sistem dijaga melalui aturan bahwa setiap event logis hanya boleh diproses satu kali. Jika event yang sama masuk berulang, sistem hanya mencatatnya sebagai duplicate dan tidak menyimpan ulang event ke tabel utama. Endpoint /stats digunakan untuk memverifikasi konsistensi dengan invariant utama: $\text{received} = \text{unique_processed} + \text{duplicate_dropped}$. Jika nilai tersebut benar dan error_count bernilai nol, maka accounting event dianggap konsisten.

Konsep ini berkaitan dengan consistency dan replication dalam distributed systems. Van Steen dan Tanenbaum (2023) membahas bahwa sistem terdistribusi sering menghadapi trade-off antara konsistensi, ketersediaan, dan performa. Pada implementasi ini, sistem tidak menerapkan replikasi database penuh, tetapi tetap menerapkan prinsip konsistensi melalui deduplication dan idempotent write. Secara praktis, sistem dapat dipandang eventually consistent pada sisi observability: setelah publisher selesai mengirim event, data event unik, audit log, dan statistik akan berakhir pada kondisi yang konsisten.

T6 Transaksi dan Kontrol Konkurensi

Transaksi digunakan untuk memastikan bahwa beberapa operasi penting diproses sebagai satu kesatuan. Ketika event diterima, sistem mencoba menyimpan event ke tabel events, mencatat audit log, dan memperbarui statistik. Jika event baru, counter unique_processed bertambah. Jika event duplikat, counter duplicate_dropped bertambah. Counter received tetap bertambah untuk semua event yang masuk. Penggunaan transaksi menjaga atomicity dan consistency agar tidak terjadi data setengah tersimpan.

Kontrol konkurensi diterapkan menggunakan unique constraint pada kombinasi topic dan event_id, serta query INSERT ... ON CONFLICT DO NOTHING. Jika dua request dengan event yang sama masuk secara bersamaan, PostgreSQL hanya mengizinkan satu insert berhasil. Request lain akan dianggap conflict dan diklasifikasikan sebagai duplicate. Ini merupakan pola **idempotent upsert**. Van Steen dan Tanenbaum (2023) membahas pentingnya koordinasi dan kontrol akses bersama dalam sistem terdistribusi, sedangkan mekanisme transaksi DBMS digunakan pada implementasi ini untuk mencegah race condition, lost-update, dan double-processing.

T7 Keamanan dan Sistem Berkas/Penyimpanan Terdistribusi

Keamanan dasar sistem diterapkan melalui isolasi jaringan Docker Compose. Hanya aggregator yang diekspos ke host melalui port 8080 untuk kebutuhan demo lokal. PostgreSQL dan Redis tetap berada dalam jaringan internal Compose, sehingga tidak perlu diakses langsung dari luar. Pembatasan akses ini mengurangi permukaan serangan dan sesuai dengan prinsip bahwa resource internal tidak seharusnya dibuka tanpa kebutuhan jelas. Van Steen dan Tanenbaum (2023) membahas security sebagai aspek penting dalam distributed systems, terutama terkait akses, trust, dan perlindungan resource.

Dari sisi penyimpanan, sistem menggunakan PostgreSQL dengan named volume. Volume ini berfungsi sebagai storage persisten agar data tetap aman walaupun container aggregator dihentikan atau dibuat ulang. Deduplication store tidak disimpan di memori aplikasi, melainkan di database. Karena itu, setelah aggregator restart, event lama tetap dikenali sebagai duplicate. Hal ini membuktikan bahwa persistensi data berjalan dan sistem tidak kehilangan state penting saat terjadi restart service.

T8 Sistem Berbasis Web dan Koordinasi Antar Service

Aggregator menyediakan sistem berbasis web melalui API FastAPI. Endpoint seperti /publish, /publish/batch, /events, /stats, /audit, dan /health digunakan sebagai antarmuka

komunikasi antara user, publisher, dan sistem penyimpanan. Endpoint /health berfungsi sebagai readiness dan liveness check, sedangkan /stats dan /audit digunakan untuk observability. Ini penting karena dalam sistem terdistribusi, monitoring diperlukan untuk memahami kondisi service yang berjalan terpisah.

Koordinasi antar service dilakukan oleh Docker Compose melalui dependency, healthcheck, network, dan volume. PostgreSQL dan Redis harus siap sebelum aggregator berjalan, dan publisher menunggu aggregator healthy sebelum mengirim event. Dengan mekanisme ini, Compose berperan sebagai mini-orchestrator untuk menjalankan sistem multi-service secara lokal. Van Steen dan Tanenbaum (2023) menjelaskan bahwa koordinasi merupakan bagian penting dalam distributed systems, terutama ketika proses yang berbeda harus bekerja dalam urutan dan kondisi yang tepat. Pada sistem ini, koordinasi tersebut terlihat dari startup service, readiness check, logging, dan endpoint observability.

Daftar Pustaka

van Steen, M., & Tanenbaum, A. S. (2023). *Distributed systems* (4th ed., Version 4.01). Maarten van Steen.